



Nama/NIM : Claudya Destine Julia Handoko/2341760008
Mata Kuliah : Pemrograman Web Lanjut (PWL)
Program Studi : D4 – Teknik Informatika / D4 – Sistem Informasi Bisnis
Semester : 4 (empat) / 6 (enam)
Pertemuan ke- : 1 (satu)

JOBSHEET 04

MODEL dan ELOQUENT ORM

Sebelumnya kita sudah membahas mengenai *Migration*, *Seeder*, *DB Façade*, *Query Builder*, dan sedikit tentang *Eloquent ORM* yang ada di Laravel. Sebelum kita masuk pada pembuatan aplikasi berbasis website, alangkah baiknya kita perlu menyiapkan Basis data sebagai tempat menyimpan data-data pada aplikasi kita nanti. Selain itu, umumnya kita perlu menyiapkan juga data awal yang kita gunakan sebelum membuat aplikasi, seperti data user administrator, data pengaturan sistem, dll.

Dalam pertemuan kali ini kita akan memahami tentang bagaimana cara menampilkan data, mengubah data, dan menghapus data menggunakan teknik Eloquent.

Sesuai dengan **Studi Kasus PWL.pdf**.

Jadi project Laravel 10 kita masih sama dengan menggunakan repositori **PWL_POS**.

Project PWL_POS akan kita gunakan sampai pertemuan 12 nanti, sebagai project yang akan kita pelajari

ORM (Object Relation Mapping) merupakan teknik yang merubah suatu table menjadi sebuah object yang nantinya mudah untuk digunakan. Object yang dibuat memiliki property yang sama dengan field — field yang ada pada table tersebut. ORM tersebut bertugas sebagai penghubung dan sekaligus mempermudah kita dalam membuat aplikasi yang menggunakan database relasional agar menjadikan tugas kita lebih efisien. **Kelebihan - Kelebihan Menggunakan ORM**



1. Terdapat banyak fitur seperti transactions, connection pooling, migrations, seeds, streams, dan lain sebagainya.
2. perintah query memiliki kinerja yang lebih baik, daripada kita menulisnya secara manual.
3. Kita menulis model data hanya di satu tempat, sehingga lebih mudah untuk update, maintain, dan reuse the code.
4. Memungkinkan kita memanfaatkan OOP (object oriented programming) dengan baik
Di Laravel sendiri telah disediakan Eloquent ORM untuk mempermudah kita dalam melakukan berbagai macam query ke database, dan membuat pekerjaan kita menjadi lebih mudah karena tidak perlu menuliskan query sql yang panjang untuk memproses data.

A. PROPERTI `$fillable` DAN `$guarded`

1. `$fillable`

Variable `$fillable` berguna untuk mendaftarkan atribut (nama kolom) yang bisa kita isi ketika melakukan insert atau update ke database. Sebelumnya kita sudah memahami menambahkan record baru ke database. Untuk langkah menambahkan Variable `$fillable` bisa dengan menambahkan *script* seperti di bawah ini pada file model

```
protected $fillable = ['level_id', 'username'];
```

Praktikum 1 - `$fillable`:

1. Buka file model dengan nama `UserModel.php` dan tambahkan `$fillable` seperti gambar di bawah ini



```
class UserModel extends Model
{
    use HasFactory;

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
    /**
     * The attributes that are mass assignable.
     *
     * @var array
     */
    protected $fillable = ['level_id', 'username', 'nama', 'password'];
}
```

Hasil :

```
app > Models > UserModel.php > PHP Intelephense > UserModel > $fillable
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  3 references | 0 implementations
9  class UserModel extends Model
10 {
11     use HasFactory;
12
13     2 references
14     protected $table = 'm_user'; //Mendefinisikan nama tabel yang digunakan oleh model ini
15     2 references
16     protected $primaryKey = 'user_id'; //Mendefinisikan primary key dari tabel yang digunakan
17
18     /**
19      * The attributes that are mass assignable
20      *
21      * @var array
22      */
23     4 references
24     protected $fillable = ['level_id', 'username', 'nama', 'password'];
25 }
```

2. Buka file controller dengan nama **UserController.php** dan ubah *script* untuk menambahkan data baru seperti gambar di bawah ini



```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use App\Models\UserModel;
7  use Illuminate\Support\Facades\Hash;
8
9  class UserController extends Controller
10 {
11     public function index()
12     {
13         $data = [
14             'level_id' => 2,
15             'username' => 'manager_dua',
16             'nama' => 'Manager 2',
17             'password' => Hash::make('12345')
18         ];
19         UserModel::create($data);
20
21         $user = UserModel::all();
22         return view('user', ['data' => $user]);
23     }
24 }
25
```

Hasil :

```
app > Http > Controllers > UserController.php > PHP Intelephense > UserController > index
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\UserModel;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Hash;
8
9  class UserController extends Controller
10 {
11     public function index(): Factory|View
12     {
13         $data = [
14             'level_id' => 2,
15             'username' => 'manager_dua',
16             'nama' => 'Manager 2',
17             'password' => Hash::make('12345')
18         ];
19
20         UserModel::create($data);
21
22         $user = UserModel::all();
23         return view(view: 'user', data: ['data' => $user]);
24     }
25
26
27
28 }
```



3. Simpan kode program Langkah 1 dan 2, dan jalankan perintah web server. Kemudian jalankan link localhostPWL_POS/public/user pada *browser* dan amati apa yang terjadi
4. Ubah file model `UserModel.php` seperti pada gambar di bawah ini pada bagian `$fillable`

```
protected $fillable = ['level_id', 'username', 'nama'];
```

Hasil :

```
app > Models > UserModel.php > PHP Intelephense > UserModel
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  3 references | 0 implementations
class UserModel extends Model
9
10 | use HasFactory;
11
12 | 2 references
protected $table = 'm_user'; //Mendefinisikan nama tabel yang digunakan oleh model ini
13 | 2 references
protected $primaryKey = 'user_id'; //Mendefinisikan primary key dari tabel yang digunakan
14
15
16 | 4 references
protected $fillable = ['level_id', 'username', 'nama'];
17
```

5. Ubah kembali file controller `UserController.php` seperti pada gambar di bawah hanya bagian array pada `$data`

```
public function index()
{
    $data = [
        'level_id' => 2,
        'username' => 'manager_tiga',
        'nama' => 'Manager 3',
        'password' => Hash::make('12345')
    ];
    UserModel::create($data);

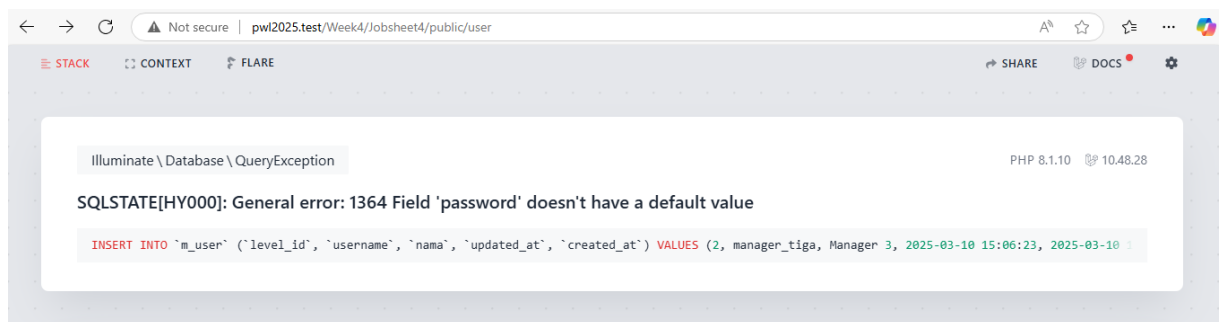
    $user = UserModel::all();
    return view('user', ['data' => $user]);
}
```

Hasil :



```
9 class UserController extends Controller
10 {
11     1 reference | 0 overrides
12     public function index(): Factory|View
13     {
14         //tambah data user 2
15         $data = [
16             'level_id' => 2,
17             'username' => 'manager_tiga',
18             'nama' => 'Manager 3',
19             'password' => Hash::make('12345')
20         ];
21         //insert data ke tabel m_user
22         UserModel::create($data);
23
24         //coba akses model UserModel
25         $user = UserModel::all(); //ambil semua data dari tabel m_user
26         return view('user', data: ['data' => $user]);
27     }
28 }
```

6. Simpan kode program Langkah 4 dan 5. Kemudian jalankan pada *browser* dan amati apa yang terjadi



7. Laporkan hasil Praktikum-¹ ini dan *commit* perubahan pada *git*.

2. `$guarded`

Kebalikan dari `$fillable` adalah `$guarded`. Semua kolom yang kita tambahkan ke `$guarded` akan diabaikan oleh Eloquent ketika kita melakukan insert/update. Secara default `$guarded` isinya `array("*")`, yang berarti semua atribut tidak bisa diset melalui **mass assignment**. **Mass Assignment** adalah fitur canggih yang menyederhanakan proses pengaturan beberapa atribut model sekaligus, menghemat waktu dan tenaga. Pada praktikum ini, kita akan mengeksplorasi konsep penugasan massal di Laravel dan bagaimana hal itu dapat dimanfaatkan secara efektif untuk meningkatkan alur kerja pengembangan Anda.



B. RETRIEVING SINGLE MODELS

Selain mengambil semua rekaman yang cocok dengan kueri tertentu, Anda juga dapat mengambil rekaman tunggal menggunakan metode `find`, `first`, atau `firstWhere`. Daripada mengembalikan kumpulan model, metode ini mengembalikan satu contoh model dan dilakukan pada controller:

```
// Ambil model dengan kunci utamanya...
$user = UserModel::find(1);

// Ambil model pertama yang cocok dengan batasan kueri...
$user = UserModel::where('level_id', 1)->first();

// Alternatif untuk mengambil model pertama yang cocok dengan batasan kueri...
$user = UserModel::firstWhere('level_id', 1);
```

Praktikum 2.1 – Retrieving Single Models

1. Buka file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::find(1);
        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
1 reference | 0 overrides
11 public function index(): Factory|View
12
13 //praktikum2 1
14
15 $user = UserModel::find(1);
16 return view(view: 'user', data: ['data' => $user]);
17
```

2. Buka file view dengan nama `user.blade.php` dan ubah *script* seperti gambar di bawah ini



```
<body>
  <h1>Data User</h1>
  <table border="1" cellpadding="2" cellspacing="0">
    <tr>
      <td>ID</td>
      <td>Username</td>
      <td>Nama</td>
      <td>ID Level Pengguna</td>
    </tr>
    <tr>
      <td>{{ $data->user_id }}</td>
      <td>{{ $data->username }}</td>
      <td>{{ $data->nama }}</td>
      <td>{{ $data->level_id }}</td>
    </tr>
  </table>
</body>
```

Hasil :

```
resources > views > user.blade.php > html > body > table > tr >
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <title>Data User</title>
5  </head>
6  <body>
7    <h1>Data User</h1>
8    <table border="1" cellpadding="2" cellspacing="0">
9      <tr>
10       <th>ID</th>
11       <th>Username</th>
12       <th>Nama</th>
13       <th>ID Level Pengguna</th>
14     </tr>
15     <tr>
16       <td>{{ $data->user_id }}</td>
17       <td>{{ $data->username }}</td>
18       <td>{{ $data->nama }}</td>
19       <td>{{ $data->level_id }}</td>
20     </tr>
21   </table>
22 </body>
23 </html>
```

3. Simpan kode program Langkah 1 dan 2. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



The screenshot shows a web browser window with the address bar displaying 'pwl2025.test/Week4/Jobsheet4/public/user'. The page title is 'Data User'. Below the title is a table with the following data:

ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1

4. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

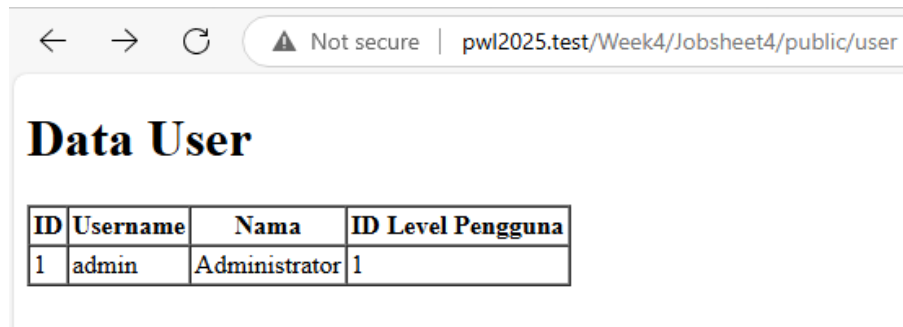
```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::where('level_id', 1)->first();
        return view('user', ['data' => $user]);
    }
}
```

Hasil :

The screenshot shows an IDE window with the file path 'app > Http > Controllers > UserController.php'. The code is as follows:

```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Models\UserModel;
6 use Illuminate\Http\Request;
7 use Illuminate\Support\Facades\Hash;
8
9 class UserController extends Controller
10 {
11     public function index(): Factory|View
12     {
13         //praktikum2 4
14
15         $user = UserModel::where('level_id', 1)->first();
16         return view(view: 'user', data: ['data' => $user]);
17     }
18 }
```

5. Simpan kode program Langkah 4. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



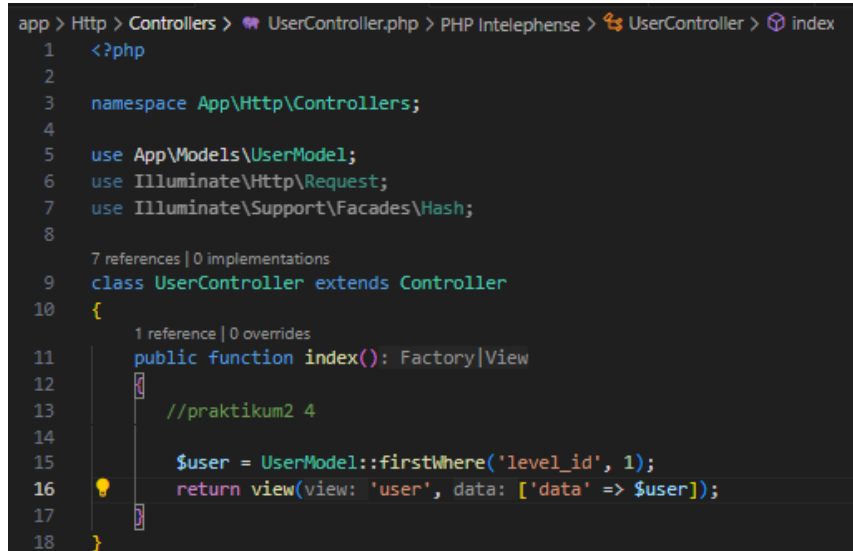
The screenshot shows a web browser window with the address bar displaying "pwl2025.test/Week4/Jobsheet4/public/user". The page title is "Data User". Below the title is a table with the following data:

ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1

6. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstWhere('level_id', 1);
        return view('user', ['data' => $user]);
    }
}
```

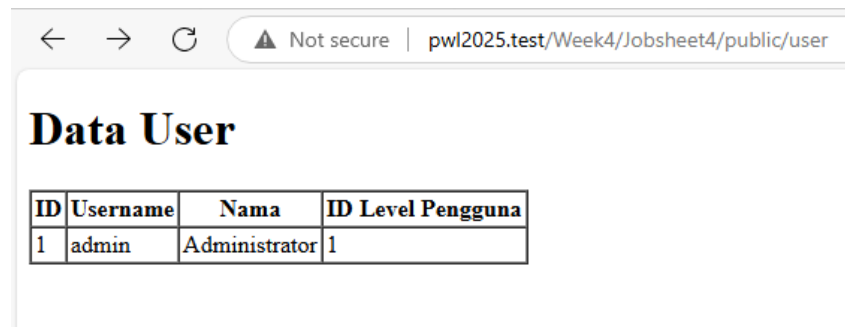
Hasil :



The screenshot shows an IDE window with the file path "app > Http > Controllers > UserController.php". The code is as follows:

```
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Models\UserModel;
6 use Illuminate\Http\Request;
7 use Illuminate\Support\Facades\Hash;
8
9 class UserController extends Controller
10 {
11     public function index(): Factory|View
12     {
13         //praktikum2 4
14
15         $user = UserModel::firstWhere('level_id', 1);
16         return view(view: 'user', data: ['data' => $user]);
17     }
18 }
```

7. Simpan kode program Langkah 6. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1

Terkadang Anda mungkin ingin melakukan beberapa tindakan lain jika tidak ada hasil yang ditemukan. Metode `findOr` and `firstOr` akan mengembalikan satu contoh model atau, jika tidak ada hasil yang ditemukan maka akan menjalankan didalam fungsi. Nilai yang dikembalikan oleh fungsi akan dianggap sebagai hasil dari metode ini:

```
$user = UserModel::findOr(1, function () {  
    // ...  
});  
  
$user = UserModel::where('level_id', '>', 3)->firstOr(function () {  
    // ...  
});
```

- Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

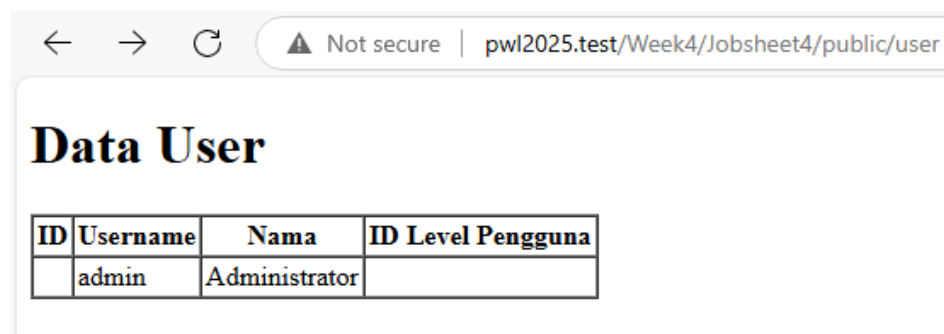
```
class UserController extends Controller  
{  
    public function index()  
    {  
        $user = UserModel::findOr(1, ['username', 'nama'], function () {  
            abort(404);  
        });  
  
        return view('user', ['data' => $user]);  
    }  
}
```

Hasil :



```
app > Http > Controllers > UserController.php > PHP > UserController > index()
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\UserModel;
6  use Illuminate\Http\Request;
7  use Illuminate\Support\Facades\Hash;
8
9  class UserController extends Controller
10 {
11     public function index(): Factory|View
12     {
13         //praktikum2 8
14
15         $user = UserModel::findOr(1, ['username', 'nama'], function () { never {
16             abort(code: 404);
17         });
18         return view(view: 'user', data: ['data' => $user]);
19     }
20 }
```

9. Simpan kode program Langkah 8. Kemudian pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



10. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::findOr(20, ['username', 'nama'], function () {
            abort(404);
        });
        return view('user', ['data' => $user]);
    }
}
```

Hasil :

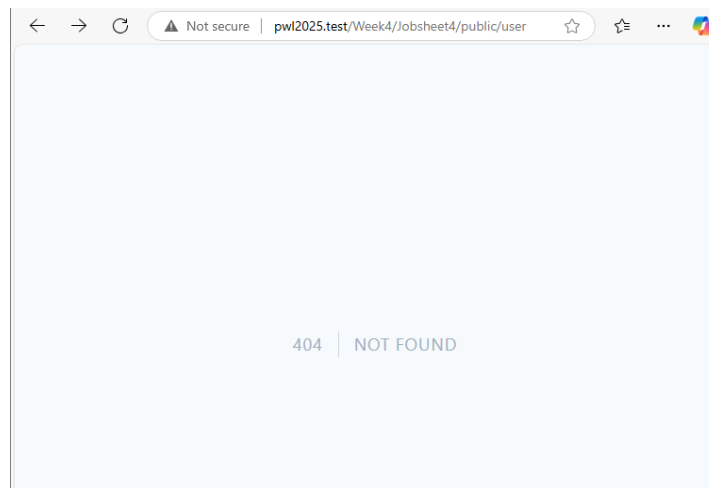


```
1 reference | 0 overrides
public function index(): Factory\View

//praktikum2 10

$user = UserModel::findOr(20, ['username', 'nama'], function () { never {
    abort(code: 404);
}});
return view(view: 'user', data: ['data' => $user]);
}
```

11. Simpan kode program Langkah 10. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



12. Laporkan hasil Praktikum-2.1 ini dan *commit* perubahan pada *git*.

Praktikum 2.2 – Not Found Exceptions

Terkadang Anda mungkin ingin memberikan pengecualian jika model tidak ditemukan. Hal ini sangat berguna dalam *route* atau pengontrol. Metode `findOrFail` and `firstOrFail` akan mengambil hasil pertama dari kueri; namun, jika tidak ada hasil yang ditemukan, sebuah `Illuminate\Database\Eloquent\ModelNotFoundException` akan dilempar. Berikut ikuti langkah-langkah di bawah ini:

1. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

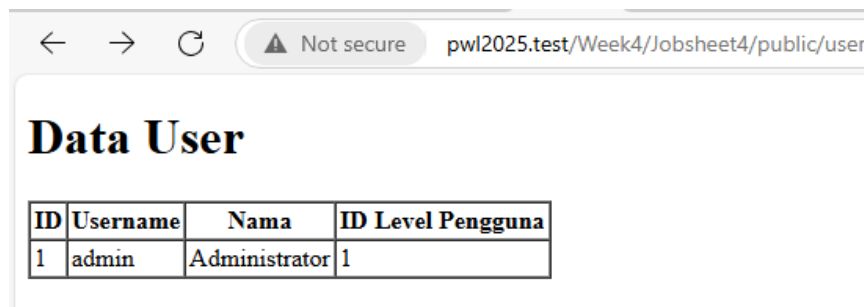


```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::findOrFail(1);
        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
9 class UserController extends Controller
10 {
11     1 reference | 0 overrides
    public function index(): Factory|View
12
13     //praktikum2.2 1
14     $user = UserModel::findOrFail(1);
15     return view(view: 'user', data: ['data' => $user]);
16
17 }
```

2. Simpan kode program Langkah 1. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



3. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

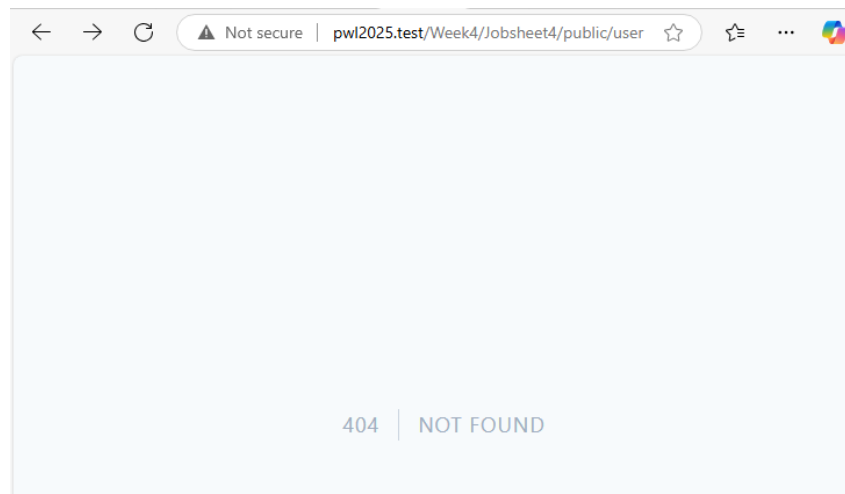
```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::where('username', 'manager9')->firstOrFail();
        return view('user', ['data' => $user]);
    }
}
```

Hasil :



```
7 references | 0 implementations
9 class UserController extends Controller
10 {
11     1 reference | 0 overrides
12     public function index(): Factory|View
13     {
14         //praktikum2.2 3|
15         $user = UserModel::where('username', 'manager9')->firstOrFail();
16         return view(view: 'user', data: ['data' => $user]);
17     }
}
```

4. Simpan kode program Langkah 3. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



5. Laporkan hasil Praktikum-2.2 ini dan *commit* perubahan pada *git*.

Praktikum 2.3 – Retrieving Aggregates

Saat berinteraksi dengan model Eloquent, Anda juga dapat menggunakan metode agregat `count`, `sum`, `max`, dan lainnya yang disediakan oleh pembuat kueri Laravel. Seperti yang Anda duga, metode ini mengembalikan nilai skalar dan contoh model Eloquent:

```
$count = UserModel::where('active', 1)->count();

$max = UserModel::where('active', 1)->max('price');
```

1. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

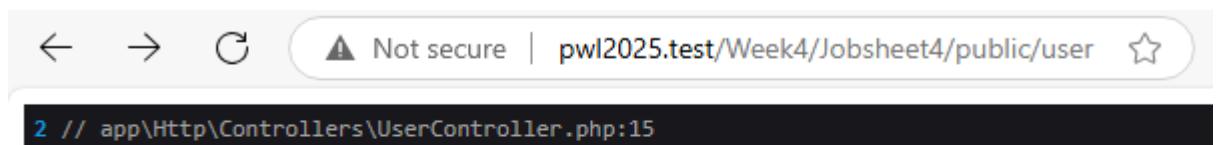


```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::where('level_id', 2)->count();
        dd($user);
        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
9 class UserController extends Controller
10 {
11     1 reference | 0 overrides
12     public function index(): Factory|View
13     {
14         //praktikum2.3 1
15         $user = UserModel::where('level_id', 2)->count();
16         dd(vars: $user);
17         return view(view: 'user', data: ['data' => $user]);
18     }
}
```

2. Simpan kode program Langkah 1. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



3. Buat agar jumlah *script* pada langkah 1 bisa tampil pada halaman *browser*, sebagai contoh bisa lihat gambar di bawah ini dan ubah *script* pada file *view* supaya bisa muncul datanya

Data User

Jumlah Pengguna
2

Jawaban :



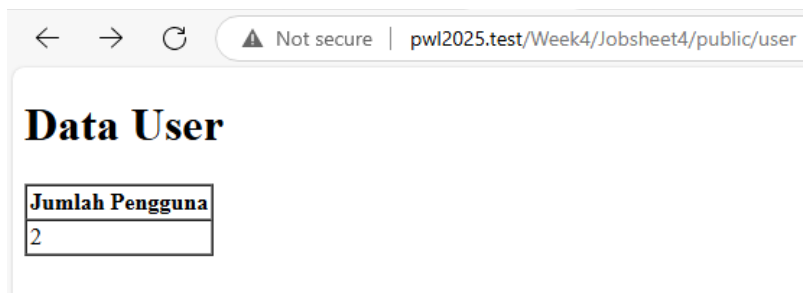
- **UserController.php**

```
//praktikum2.3 3
$user = UserModel::where('level_id', 2)->count();
return view(view: 'user', data: ['data' => $user]);
```

- **User.blade.php**

```
resources > views > user.blade.php > html > body > table > tr > td
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 | <title>Data User</title>
5 </head>
6 <body>
7 | <h1>Data User</h1>
8 | <table border="1" cellpadding="2" cellspacing="0">
9 | | <tr>
10 | | | <th>Jumlah Pengguna</th>
11 | | </tr>
12 | | <tr>
13 | | | <td>{{ $data }}</td>
14 | | </tr>
15 | </table>
16 </body>
17 </html>
```

- **Hasil**



4. Laporkan hasil Praktikum-2.3 ini dan *commit* perubahan pada *git*.

Praktikum 2.4 – Retrieving or Creating Models

Metode `firstOrCreate` merupakan metode untuk melakukan *retrieving data* (mengambil data) berdasarkan nilai yang ingin dicari, jika data tidak ditemukan maka method ini akan melakukan insert ke table database tersebut sesuai dengan nilai yang dimasukkan.

Metode `firstOrCreate`, seperti `firstOrCreate`, akan mencoba menemukan/mengambil *record/data* dalam database yang cocok dengan atribut yang diberikan. Namun, jika data tidak



ditemukan, data akan disiapkan untuk di-*insert*-kan ke database dan model baru akan dikembalikan. Perhatikan bahwa model yang dikembalikan `firstOrCreate` belum disimpan ke database. Anda perlu memanggil metode `save()` secara manual untuk menyimpannya:

```
$user = UserModel::firstOrCreate([
    'username' => 'manager',
    'nama' => 'Manager',
]);

$user = UserModel::firstOrCreate([
    'username' => 'manager',
    'nama' => 'Manager',
]);
```

1. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate([
            'username' => 'manager',
            'nama' => 'Manager',
        ]);

        return view('user', ['data' => $user]);
    }
}
```

Hasil :



```
public function index(): Factory|View  
  
    //praktikum2.4 1  
    $user = UserModel::firstOrCreate(  
        [  
            'username' => 'manager',  
            'nama' => 'Manager',  
        ],  
    );  
    return view('user', data: ['data' => $user]);
```



2. Ubah kembali file view dengan nama `user.blade.php` dan ubah *script* seperti gambar di bawah ini

```
<body>
<h1>Data User</h1>
<table border="1" cellpadding="2" cellspacing="0">
  <tr>
    <td>ID</td>
    <td>Username</td>
    <td>Nama</td>
    <td>ID Level Pengguna</td>
  </tr>
  <tr>
    <td>{{ $data->user_id }}</td>
    <td>{{ $data->username }}</td>
    <td>{{ $data->nama }}</td>
    <td>{{ $data->level_id }}</td>
  </tr>
</table>
</body>
```

Hasil :

```
6 <body>
7   <h1>Data User</h1>
8   <table border="1" cellpadding="2" cellspacing="0">
9     <tr>
10      <th>ID</th>
11      <th>Username</th>
12      <th>Nama</th>
13      <th>ID Level Pengguna</th>
14    </tr>
15    <tr>
16      <td>{{ $data->user_id }}</td>
17      <td>{{ $data->username }}</td>
18      <td>{{ $data->nama }}</td>
19      <td>{{ $data->level_id }}</td>
20    </tr>
21  </table>
22 </body>
23 </html>
```

3. Simpan kode program Langkah 1 dan 2. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



← → ↻ Not secure | pwl2025.test/Week4/Jobsheet4/public/user

Data User

ID	Username	Nama	ID Level Pengguna
2	Manager	Manager	2

4. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager22',
                'nama' => 'Manager Dua Dua',
                'password' => Hash::make('12345'),
                'level_id' => 2
            ],
        );

        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
public function index(): Factory|View
{
    //praktikum2.4 4
    $user = UserModel::firstOrCreate(
        [
            'username' => 'manager22',
            'nama' => 'Manager Dua Dua',
            'password' => Hash::make('12345'),
            'level_id' => 2
        ],
    );
    return view('user', data: ['data' => $user]);
}
```

Tambahan UserModel.php

```
4 references
protected $fillable = ['level_id', 'username', 'nama', 'password'];
```



5. Simpan kode program Langkah 4. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan cek juga pada *phpMyAdmin* pada tabel `m_user` serta beri penjelasan dalam laporan

ID	Username	Nama	ID Level Pengguna
12	manager22	Manager Dua Dua	2

6. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager',
                'nama' => 'Manager',
            ],
        );

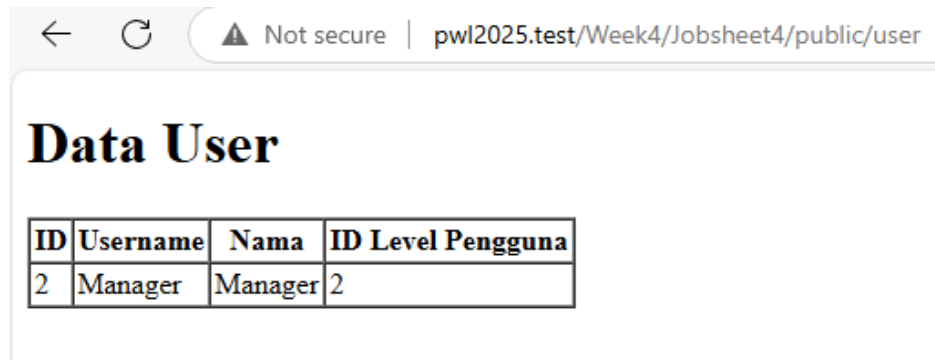
        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
//praktikum2.4 6
$user = UserModel::firstOrCreate(
    [
        'username' => 'manager',
        'nama' => 'Manager',
    ],
);
return view(view: 'user', data: ['data' => $user]);
```



7. Simpan kode program Langkah 6. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



8. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager33',
                'nama' => 'Manager Tiga Tiga',
                'password' => Hash::make('12345'),
                'level_id' => 2
            ],
        );

        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
public function index(): Factory|View
{
    //praktikum2.4 8
    $user = UserModel::firstOrCreate(
        [
            'username' => 'manager33',
            'nama' => 'Manager Tiga Tiga',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ],
    );

    return view('user', data: ['data' => $user]);
}
```



9. Simpan kode program Langkah 8. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan cek juga pada *phpMyAdmin* pada tabel *m_user* serta beri penjelasan dalam laporan

The top screenshot shows a web browser at the URL `pwl2025.test/Week4/Jobsheet4/public/user`. The page title is "Data User" and it displays a table with the following data:

ID	Username	Nama	ID Level Pengguna
	manager33	Manager Tiga Tiga	2

The bottom screenshot shows the phpMyAdmin interface for the `m_user` table. The table structure is as follows:

	user_id	level_id	username	nama	password
☐ Edit ☐ Copy ☐ Delete	1	1	admin	Administrator	\$2y\$12\$4yXRzPmKWJqra5OBAmxpu6r8JmUWc5wP.c9R.
☐ Edit ☐ Copy ☐ Delete	2	2	Manager	Manager	\$2y\$12\$aA.KKkeD1Sd249ot5IFH8uQTO0/DGZ4Ly4YavnD:
☐ Edit ☐ Copy ☐ Delete	3	3	Staff	Staff/Kasir	\$2y\$12\$vjUxiaoYHkSy/AkcjKF62.vQtToUzhFv46bDoOG/Ux
☐ Edit ☐ Copy ☐ Delete	4	4	customer-1	Manager 1	\$2y\$12\$qEuE1/c5PgZ7tzhfUioLOqyxFQYHO.Gi.tXQlga/Ec
☐ Edit ☐ Copy ☐ Delete	5	2	manager_dua	Manager 2	\$2y\$12\$vx:fMjK65IWygkMLIGBI1wufOSbaeDCbtJoMWuYrE
☐ Edit ☐ Copy ☐ Delete	12	2	manager22	Manager Dua Dua	\$2y\$12\$G0VAZuNWOKYUT5YJekHwrewOpNouAVbZOF16

Keterangan : `firstOrCreate()` untuk mencari data pengguna berdasarkan username = 'manager33'. Jika tidak ditemukan, Laravel akan membuat objek baru tetapi belum menyimpannya ke database. Karena tidak ada perintah `$user->save();`, data tidak akan tersimpan permanen di database. Data kemudian dikirim ke tampilan user untuk ditampilkan di halaman web.

10. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini



```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::firstOrCreate(
            [
                'username' => 'manager33',
                'nama' => 'Manager Tiga Tiga',
                'password' => Hash::make('12345'),
                'level_id' => 2
            ],
        );
        $user->save();

        return view('user', ['data' => $user]);
    }
}
```

Hasil :

```
public function index(): Factory|View
{
    //praktikum2.4 10
    $user = UserModel::firstOrCreate(
        [
            'username' => 'manager33',
            'nama' => 'Manager Tiga Tiga',
            'password' => Hash::make('12345'),
            'level_id' => 2
        ],
    );
    $user->save();

    return view('user', data: ['data' => $user]);
}
```

11. Simpan kode program Langkah 9. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan cek juga pada *phpMyAdmin* pada tabel *m_user* serta beri penjelasan dalam laporan

← ↻ ⚠ Not secure | pwl2025.test/Week4/Jobsheet4/public/...

Data User

ID	Username	Nama	ID Level Pengguna
13	manager33	Manager Tiga Tiga	2



				user_id	level_id	username	nama	password
☐				1	1	admin	Administrator	\$2y\$12\$4yXRzPmKWJqra5OBAmxpu8r8JmUW65wP.c6
☐				2	2	Manager	Manager	\$2y\$12\$aA.KKkeD1Sd249ot5lFH6uQTO0/DGZ4Ly4Yavr
☐				3	3	Staff	Staff/Kasir	\$2y\$12\$vjUxiaoYHkSy/AkcjKF62.vQtToUzhFv48bDoOG/
☐				4	4	customer-1	Manager 1	\$2y\$12\$qEuE1/c5PgZ7ttzhfUioLQqyxFQYHO.Gi.tXQlga/
☐				5	2	manager_dua	Manager 2	\$2y\$12\$vxMjk85lWygkMLIGBI1wufOSbaeDCbtJoMWuY
☐				12	2	manager22	Manager Dua Dua	\$2y\$12\$G0VAZuNWOkYUT5YJekHwrewOpNouAVbZOF
☐				13	2	manager33	Manager Tiga Tiga	\$2y\$12\$QUQzsmYnwFw3EkfYJ2nX8.xN1qrapx1P3CqV1

12. Laporkan hasil Praktikum-2.4 ini dan *commit* perubahan pada *git*.

Praktikum 2.5 – Attribute Changes

Eloquent menyediakan metode `isDirty`, `isClean`, dan `wasChanged` untuk memeriksa keadaan internal model Anda dan menentukan bagaimana atributnya berubah sejak model pertama kali diambil.

Metode `isDirty` menentukan apakah ada atribut model yang telah diubah sejak model diambil. Anda dapat meneruskan nama atribut tertentu atau serangkaian atribut ke metode `isDirty` untuk menentukan apakah ada atribut yang "kotor". Metode ini `isClean` akan menentukan apakah suatu atribut tetap tidak berubah sejak model diambil. Metode ini juga menerima argumen atribut opsional:



```
$user = UserModel::create([
    'username' => 'manager44',
    'nama' => 'Manager44',
    'password' => Hash::make('12345'),
    'level_id' => 2,
]);

$user->username = 'manager45';

$user->isDirty(); // true
$user->isDirty('username'); // true
$user->isDirty('nama'); // false
$user->isDirty(['nama', 'username']); // true

$user->isClean(); // false
$user->isClean('username'); // false
$user->isClean('nama'); // true
$user->isClean(['nama', 'username']); // false

$user->save();

$user->isDirty(); // false
$user->isClean(); // true
```

1. Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::create([
            'username' => 'manager55',
            'nama' => 'Manager55',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ]);

        $user->username = 'manager56';

        $user->isDirty(); // true
        $user->isDirty('username'); // true
        $user->isDirty('nama'); // false
        $user->isDirty(['nama', 'username']); // true

        $user->isClean(); // false
        $user->isClean('username'); // false
        $user->isClean('nama'); // true
        $user->isClean(['nama', 'username']); // false

        $user->save();

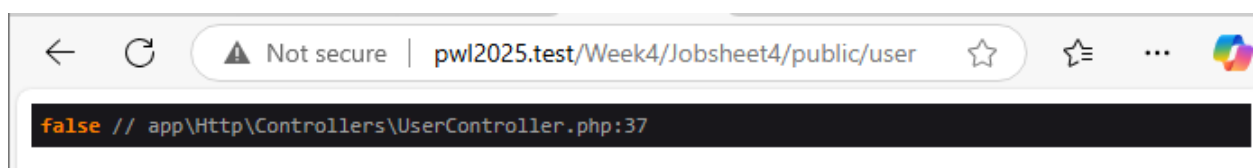
        $user->isDirty(); // false
        $user->isClean(); // true
        dd($user->isDirty());
    }
}
```



Hasil :

```
public function index(): never  
  
//praktikum2.5 1  
$user = UserModel::create([  
    'username' => 'manager55',  
    'nama' => 'Manager 55',  
    'password' => Hash::make('12345'),  
    'level_id' => 2  
]);  
  
$user->username = 'manager56';  
  
$user->isDirty(); //true  
$user->isDirty('username'); // true  
$user->isDirty('nama'); // false  
$user->isDirty(['nama', 'username']); // true  
  
$user->isClean(); // false  
$user->isClean('username'); // false  
$user->isClean('nama'); // true  
$user->isClean(['nama', 'username']); // false  
  
$user->save();  
  
$user->isDirty(); // false  
$user->isClean(); //true  
dd(vars: $user->isDirty());
```

2. Simpan kode program Langkah 1. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



Metode ini `wasChanged` menentukan apakah ada atribut yang diubah saat model terakhir disimpan dalam siklus permintaan saat ini. Jika diperlukan, Anda dapat memberikan nama atribut untuk melihat apakah atribut tertentu telah diubah:



```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::create([
            'username' => 'manager11',
            'nama' => 'Manager11',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ]);

        $user->username = 'manager12';

        $user->save();

        $user->wasChanged(); // true
        $user->wasChanged('username'); // true
        $user->wasChanged(['username', 'level_id']); // true
        $user->wasChanged('nama'); // false
        $user->wasChanged(['nama', 'username']); // true
    }
}
```

Hasil :

```
public function index(): never
{
    //praktikum2.5 2
    $user = UserModel::create([
        'username' => 'manager11',
        'nama' => 'Manager 11',
        'password' => Hash::make('12345'),
        'level_id' => 2
    ]);

    $user->username = 'manager12';

    $user->save();

    $user->wasChanged(); // true
    $user->wasChanged('username'); // true
    $user->wasChanged(['username', 'level_id']); // true
    $user->wasChanged('nama'); // false
    $user->wasChanged(['nama', 'username']); // true
}
```

- Ubah file controller dengan nama `UserController.php` dan ubah *script* seperti gambar di bawah ini



```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::create([
            'username' => 'manager11',
            'nama' => 'Manager11',
            'password' => Hash::make('12345'),
            'level_id' => 2,
        ]);

        $user->username = 'manager12';

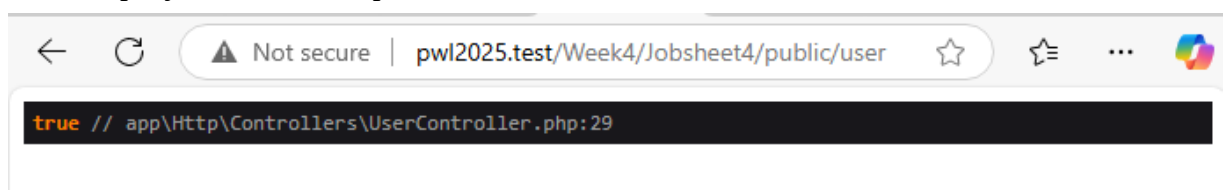
        $user->save();

        $user->wasChanged(); // true
        $user->wasChanged('username'); // true
        $user->wasChanged(['username', 'level_id']); // true
        $user->wasChanged('nama'); // false
        dd($user->wasChanged(['nama', 'username'])); // true
    }
}
```

Hasil :

```
$user->wasChanged(); // true
$user->wasChanged('username'); // true
$user->wasChanged(['username', 'level_id']); // true
$user->wasChanged('nama'); // false
dd($user->wasChanged(['nama', 'username'])); // true
```

4. Simpan kode program Langkah 3. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



5. Laporkan hasil Praktikum-2.5 ini dan *commit* perubahan pada *git*.

Praktikum 2.6 – Create, Read, Update, Delete (CRUD)

Seperti yang telah kita ketahui, CRUD merupakan singkatan dari *Create*, *Read*, *Update* dan *Delete*. CRUD merupakan istilah untuk proses pengolahan data pada database, seperti input data ke database, menampilkan data dari database, mengedit data pada database dan menghapus



data dari database. Ikuti langkah-langkah di bawah ini untuk melakukan CRUD dengan Eloquent

1. Buka file view pada `user.blade.php` dan buat scripnya menjadi seperti di bawah ini

```
<body>
<h1>Data User</h1>
<a href="/user/tambah">+ Tambah User</a>
<table border="1" cellpadding="2" cellspacing="0">
  <tr>
    <td>ID</td>
    <td>Username</td>
    <td>Nama</td>
    <td>ID Level Pengguna</td>
    <td>Aksi</td>
  </tr>
  @foreach ($data as $d)
    <tr>
      <td>{{ $d->user_id }}</td>
      <td>{{ $d->username }}</td>
      <td>{{ $d->nama }}</td>
      <td>{{ $d->level_id }}</td>
      <td><a href="/user/ubah/{{ $d->user_id }}">Ubah</a> | <a href="/user/hapus/{{ $d->user_id }}">Hapus</a></td>
    </tr>
  @endforeach
</table>
</body>
```

Hasil :

```
resources > views > user.blade.php > html > body > table > tr > td
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Data User</title>
5 </head>
6 <body>
7   <h1>Data User</h1>
8   <a href="/user/tambah">+ Tambah User</a>
9   <table border="1" cellpadding="2" cellspacing="0">
10     <tr>
11       <th>ID</th>
12       <th>Username</th>
13       <th>Nama</th>
14       <th>ID Level Pengguna</th>
15       <th>Aksi</th>
16     </tr>
17     @foreach ($data as $d )
18       <tr>
19         <td>{{ $d->user_id }}</td>
20         <td>{{ $d->username }}</td>
21         <td>{{ $d->nama }}</td>
22         <td>{{ $d->level_id }}</td>
23         <td><a href="/user/ubah/{{ $d->user_id }}">Ubah</a> | <a href="/user/hapus/{{ $d->user_id }}">Hapus</a></td>
24       </tr>
25     @endforeach
26   </table>
27 </body>
28 </html>
```

2. Buka file controller pada `UserController.php` dan buat scriptnya untuk *read* menjadi seperti di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```



Hasil :

```
public function index(): never  
  
    //praktikum2.6 2  
    $user = UserModel::all();  
    return view('user', ['data' => $user]);
```

3. Simpan kode program Langkah 1 dan 2. Kemudian jalankan pada *browser* dan amati apa yang terjadi dan beri penjelasan dalam laporan



Data User

[+ Tambah User](#)

ID	Username	Nama	ID Level Pengguna	Aksi
1	admin	Administrator	1	Ubah Hapus
2	Manager	Manager	2	Ubah Hapus
3	Staff	Staff/Kasir	3	Ubah Hapus
4	customer-1	Manager 1	4	Ubah Hapus
5	manager_dua	Manager 2	2	Ubah Hapus
12	manager22	Manager Dua Dua	2	Ubah Hapus
13	manager33	Manager Tiga Tiga	2	Ubah Hapus
14	manager56	Manager 55	2	Ubah Hapus
15	manager12	Manager 11	2	Ubah Hapus
16	manager11	Manager 11	2	Ubah Hapus

4. Langkah berikutnya membuat *create* atau tambah data user dengan cara bikin file baru pada view dengan nama `user_tambah.blade.php` dan buat scriptnya menjadi seperti di bawah ini



```
<body>
  <h1>Form Tambah Data User</h1>
  <form method="post" action="/user/tambah_simpan">

    {{ csrf_field() }}

    <label>Username</label>
    <input type="text" name="username" placeholder="Masukan Username">
    <br>
    <label>Nama</label>
    <input type="text" name="nama" placeholder="Masukan Nama">
    <br>
    <label>Password</label>
    <input type="password" name="password" placeholder="Masukan Password">
    <br>
    <label>Level ID</label>
    <input type="number" name="level_id" placeholder="Masukan ID Level">
    <br><br>
    <input type="submit" class="btn btn-success" value="Simpan">

  </form>
</body>
```

Hasil :

```
PS C:\laragon\www\PWL2025\Week4\Jobsheet4> touch resources/views/user_tambah.blade.php
```

```
resources > views > user_tambah.blade.php > body > form
1  <body>
2    <h1>Form Tambah Data User</h1>
3    <form method="post" action="/user/tambah_simpan">
4
5      {{ csrf_field() }}
6
7      <label>Username</label>
8      <input type="text" name="username" placeholder="Masukkan Username">
9      <br><br>
10
11     <label>Nama</label>
12     <input type="text" name="nama" placeholder="Masukkan Nama">
13     <br><br>
14
15     <label>Password</label>
16     <input type="password" name="password" placeholder="Masukkan Password">
17     <br><br>
18
19     <label>Level ID</label>
20     <input type="number" name="level_id" placeholder="Masukkan ID Level">
21     <br><br>
22     <input type="submit" class="btn btn-success" value="Simpan">
23
24   </form>
25 </body>
```



5. Tambahkan *script* pada *routes* dengan nama file `web.php`. Tambahkan seperti gambar di bawah ini

```
Route::get('/user/tambah', [UserController::class, 'tambah']);
```

Hasil :

```
Route::get('/user/tambah', [UserController::class, 'tambah']);
```

6. Tambahkan *script* pada controller dengan nama file *UserController.php*. Tambahkan *script* dalam class dan buat method baru dengan nama *tambah* dan diletakan di bawah method *index* seperti gambar di bawah ini

```
class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }

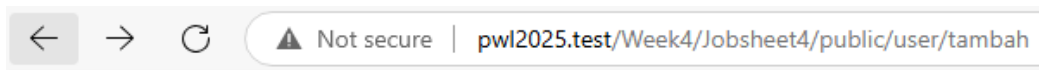
    public function tambah()
    {
        return view('user_tambah');
    }
}
```

Hasil :

```
public function index(): Factory|View
{
    //praktikum2.6 6
    $user = UserModel::all();
    return view(view: 'user', data: ['data' => $user]);
}

public function tambah()
{
    return view('user_tambah');
}
```

7. Simpan kode program Langkah 4 s/d 6. Kemudian jalankan pada *browser* dan klik link “+ **Tambah User**” amati apa yang terjadi dan beri penjelasan dalam laporan



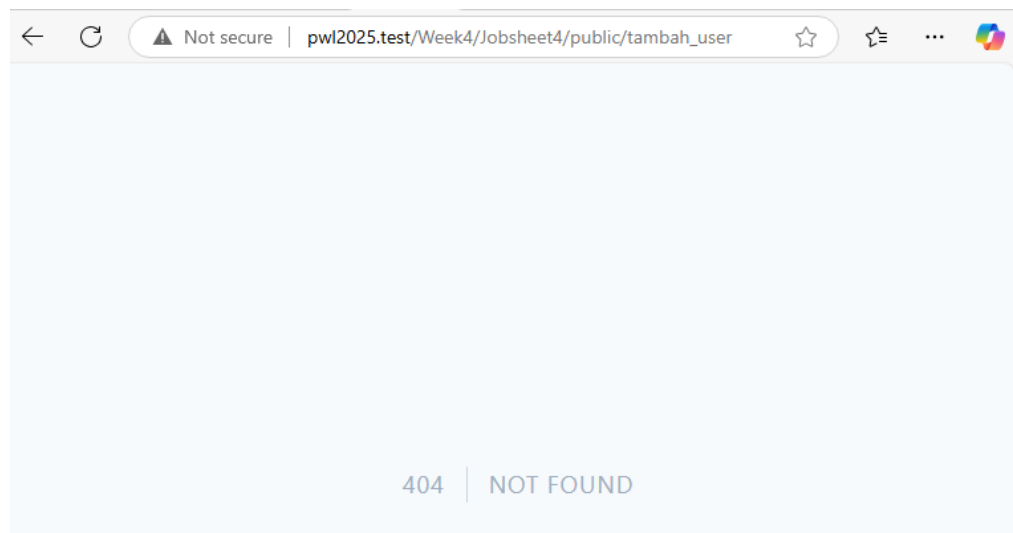
Form Tambah Data User

Username

Nama

Password

Level ID



8. Tambahkan *script* pada *routes* dengan nama file `web.php`. Tambahkan seperti gambar di bawah ini

```
Route::post('/user/tambah_simpan', [UserController::class, 'tambah_simpan']);
```

Hasil :

```
Route::post('/user/tambah_simpan', [UserController::class, 'tambah_simpan']);
```

9. Tambahkan *script* pada controller dengan nama file `UserController.php`. Tambahkan *script* dalam class dan buat method baru dengan nama `tambah_simpan` dan diletakan di bawah method `tambah` seperti gambar di bawah ini



```
public function tambah_simpan(Request $request)
{
    UserModel::create([
        'username' => $request->username,
        'nama' => $request->nama,
        'password' => Hash::make('$request->password'),
        'level_id' => $request->level_id
    ]);

    return redirect('/user');
}
```

Hasil :

```
public function tambah_simpan(Request $request): Factory|View
{
    //praktikum2.6 9
    UserModel::create([
        'username' => $request->username,
        'nama' => $request->nama,
        'password' => Hash::make('$request->password'),
        'level_id' => $request->level_id
    ]);
    return redirect('/user');
}
```

10. Simpan kode program Langkah 8 dan 9. Kemudian jalankan link <localhost:8000/user/tambah> atau localhost/PWL_POS/public/user/tambah pada browser dan input formnya dan simpan, kemudian amati apa yang terjadi dan beri penjelasan dalam laporan

← → ↻ ⚠ Not secure | pwl2025.test/Week4/Jobsheet4/public/user/tambah

Form Tambah Data User

Username

Nama

Password

Level ID



← ↻ ⚠ Not secure | pwl2025.test/Week4/Jobsheet4/public/user

Data User

[+ Tambah User](#)

ID	Username	Nama	ID Level Pengguna	Aksi
1	admin	Administrator	1	Ubah Hapus
2	Manager	Manager	2	Ubah Hapus
3	Staff	Staff/Kasir	3	Ubah Hapus
4	customer-1	Manager 1	4	Ubah Hapus
5	manager_dua	Manager 2	2	Ubah Hapus
12	manager22	Manager Dua Dua	2	Ubah Hapus
13	manager33	Manager Tiga Tiga	2	Ubah Hapus
14	manager56	Manager 55	2	Ubah Hapus
15	manager12	Manager 11	2	Ubah Hapus
16	manager11	Manager 11	2	Ubah Hapus
17	piyo	Claudya Destine	1	Ubah Hapus

11. Langkah berikutnya membuat *update* atau ubah data user dengan cara bikin file baru pada view dengan nama `user_ubah.blade.php` dan buat scriptnya menjadi seperti di bawah ini

```
<body>
<h1>Form Ubah Data User</h1>
<a href="/user">Kembali</a>
<br><br>

<form method="post" action="/user/ubah_simpan/{{ $data->user_id }}">

    {{ csrf_field() }}
    {{ method_field('PUT') }}

    <label>Username</label>
    <input type="text" name="username" placeholder="Masukan Username" value="{{ $data->username }}">
    <br>
    <label>Nama</label>
    <input type="text" name="nama" placeholder="Masukan Nama" value="{{ $data->username }}">
    <br>
    <label>Password</label>
    <input type="password" name="password" placeholder="Masukan Password" value="{{ $data->password }}">
    <br>
    <label>Level ID</label>
    <input type="number" name="level_id" placeholder="Masukan ID Level" value="{{ $data->level_id }}">
    <br><br>
    <input type="submit" class="btn btn-success" value="Ubah">

</form>
</body>
```

Hasil :



```
PS C:\laragon\www\PWL2025\Week4\Jobsheet4> touch resources/views/user_ubah.blade.php

resources > views > user_ubah.blade.php > body
1 <body>
2 <h1>Form Ubah Data User</h1>
3 <a href="/user">Kembali</a>
4 <br><br>
5
6 <form method="POST" action="/user/ubah_simpan/{{ $data->user_id }}">
7     {{ csrf_field() }}
8     {{ method_field('PUT') }}
9
10    <label>Username</label>
11    <input type="text" name="username" placeholder="Masukkan Username" value="{{ $data->username }}">
12    <br>
13    <label>Nama</label>
14    <input type="text" name="nama" placeholder="Masukkan Nama" value="{{ $data->nama }}">
15    <br>
16    <label>Password</label>
17    <input type="password" name="password" placeholder="Masukkan Password">
18    <br>
19    <label>Level ID</label>
20    <input type="number" name="level_id" placeholder="Masukkan ID Level" value="{{ $data->level_id }}">
21    <br><br>
22    <input type="submit" class="btn btn-success" value="Simpan">
23 </form>
24 </body>
25 </html>
```

12. Tambahkan *script* pada *routes* dengan nama file `web.php`. Tambahkan seperti gambar di bawah ini

```
Route::get('/user/ubah/{id}', [UserController::class, 'ubah']);
```

Hasil :

```
Route::get('/user/ubah/{id}', [UserController::class, 'ubah']);
```

13. Tambahkan *script* pada *controller* dengan nama file `UserController.php`. Tambahkan *script* dalam class dan buat method baru dengan nama `ubah` dan diletakkan di bawah method `tambah_simpan` seperti gambar di bawah ini

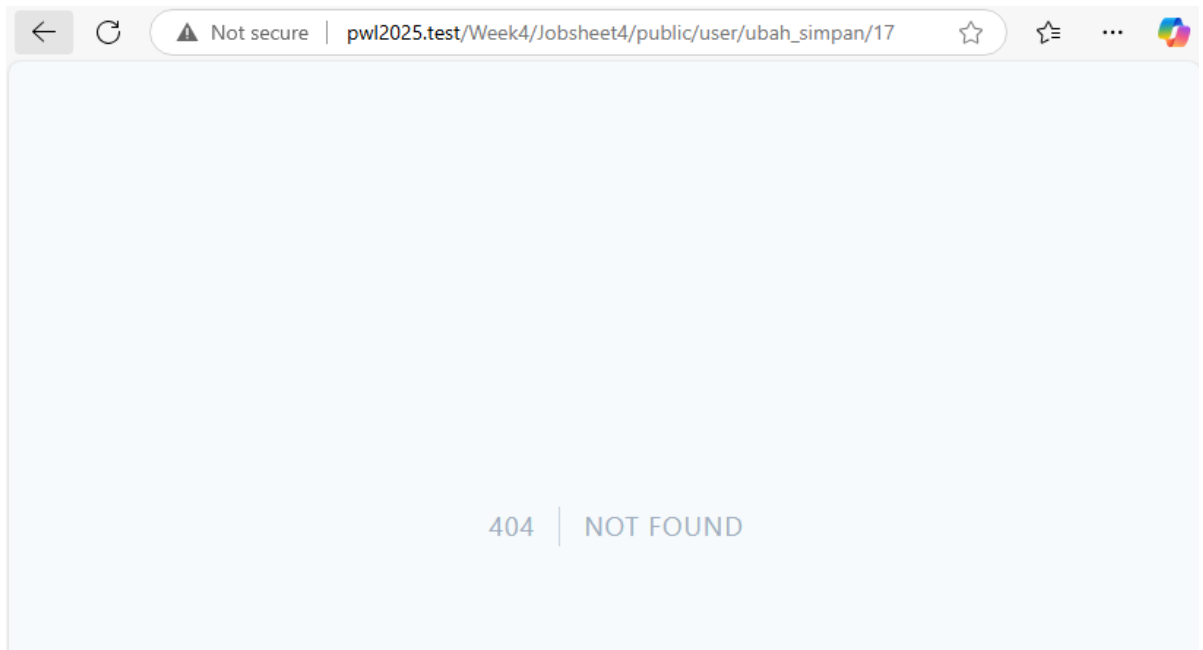
```
public function ubah($id)
{
    $user = UserModel::find($id);
    return view('user_ubah', ['data' => $user]);
}
```

Hasil :

```
//praktikum2.6 13
public function ubah($id): Factory|View
{
    $user = UserModel::find($id);
    return view(view: 'user_ubah', data: ['data' => $user]);
}
```




14. Simpan kode program Langkah 11 sd 13. Kemudian jalankan pada *browser* dan klik link “Ubah” amati apa yang terjadi dan beri penjelasan dalam laporan



15. Tambahkan *script* pada *routes* dengan nama file `web.php`. Tambahkan seperti gambar di bawah ini

```
Route::put('/user/ubah_simpan/{id}', [UserController::class, 'ubah_simpan']);
```

Hasil :

```
Route::put('/user/ubah_simpan/{id}', [UserController::class, 'ubah_simpan']);
```

16. Tambahkan *script* pada controller dengan nama file `UserController.php`. Tambahkan *script* dalam class dan buat method baru dengan nama `ubah_simpan` dan diletakan di bawah method `ubah` seperti gambar di bawah ini

```
public function ubah_simpan($id, Request $request)
{
    $user = UserModel::find($id);

    $user->username = $request->username;
    $user->nama = $request->nama;
    $user->password = Hash::make('$request->password');
    $user->level_id = $request->level_id;

    $user->save();

    return redirect('/user');
}
```

Hasil :



```
//praktikum2.6 16
1 reference | 0 overrides
public function ubah_simpan(Request $request, $id): RedirectResponse
{
    $user = UserModel::find($id);

    $user->username = $request->input('username');
    $user->nama = $request->input('nama');
    $user->password = Hash::make($request->password);
    $user->level_id = $request->level_id;

    $user->save();

    return redirect(to: '/user');
}
```

17. Simpan kode program Langkah 15 dan 16. Kemudian jalankan link <localhost:8000/user/ubah/1> atau localhost/PWL_POS/public/user/ubah/1 pada browser dan ubah input formnya dan klik tombol ubah, kemudian amati apa yang terjadi dan beri penjelasan dalam laporan

← ↻ ⓘ 127.0.0.1:8000/user/ubah/1

Form Ubah Data User

[Kembali](#)

Username

Nama

Password

Level ID

- Merubah



Form Ubah Data User

[Kembali](#)

Username

Nama

Password

Level ID

- Hasil setelah dirubah

← ↻ ⓘ 127.0.0.1:8000/user

Data User

[+ Tambah User](#)

ID	Username	Nama	ID Level Pengguna	Aksi
1	Claudya	Administrator	1	Ubah Hapus
2	Manager	Manager	2	Ubah Hapus
3	Staff	Staff/Kasir	3	Ubah Hapus
4	customer-1	Manager 1	4	Ubah Hapus
5	manager_dua	Manager 2	2	Ubah Hapus
12	manager22	Manager Dua Dua	2	Ubah Hapus
13	manager33	Manager Tiga Tiga	2	Ubah Hapus
14	manager56	Manager 55	2	Ubah Hapus
15	manager12	Manager 11	2	Ubah Hapus
16	manager11	Manager 11	2	Ubah Hapus
17	piyo	Claudya Destine	1	Ubah Hapus

18. Berikut untuk langkah *delete* . Tambahkan *script* pada *routes* dengan nama file [web.php](#).
Tambahkan seperti gambar di bawah ini

```
Route::get('/user/hapus/{id}', [UserController::class, 'hapus']);
```

Hasil :

```
Route::get('/user/hapus/{id}', [UserController::class, 'hapus']);
```

19. Tambahkan *script* pada controller dengan nama file [UserController.php](#). Tambahkan *script* dalam class dan buat method baru dengan nama *hapus* dan diletakan di bawah method *ubah_simpan* seperti gambar di bawah ini



```
public function hapus($id)
{
    $user = UserModel::find($id);
    $user->delete();

    return redirect('/user');
}
```

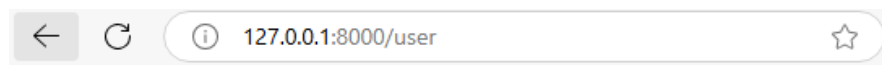
Hasil :

```
//praktikum2.6 19
1 reference | 0 overrides
public function hapus($id): Redirector|R
{
    $user = UserModel::find($id);
    $user->delete();

    return redirect(to: '/user');
}
```

20. Simpan kode program Langkah 18 dan 19. Kemudian jalankan pada *browser* dan klik tombol hapus, kemudian amati apa yang terjadi dan beri penjelasan dalam laporan

- **Tampilan awal**



Data User

[+ Tambah User](#)

ID	Username	Nama	ID Level Pengguna	Aksi
1	Claudya	Administrator	1	Ubah Hapus
2	Manager	Manager	2	Ubah Hapus
3	Staff	Staff/Kasir	3	Ubah Hapus
4	customer-1	Manager 1	4	Ubah Hapus
5	manager_dua	Manager 2	2	Ubah Hapus
12	manager22	Manager Dua Dua	2	Ubah Hapus
13	manager33	Manager Tiga Tiga	2	Ubah Hapus
14	manager56	Manager 55	2	Ubah Hapus
15	manager12	Manager 11	2	Ubah Hapus
16	manager11	Manager 11	2	Ubah Hapus
17	piyo	Claudya Destine	1	Ubah Hapus

- **Setelah dilakukan penghapusan**



← ↻ ⓘ 127.0.0.1:8000/user

Data User

[+ Tambah User](#)

ID	Username	Nama	ID Level Pengguna	Aksi
1	Claudia	Administrator	1	Ubah Hapus
2	Manager	Manager	2	Ubah Hapus
3	Staff	Staff/Kasir	3	Ubah Hapus
4	customer-1	Manager 1	4	Ubah Hapus
5	manager_dua	Manager 2	2	Ubah Hapus
12	manager22	Manager Dua Dua	2	Ubah Hapus
13	manager33	Manager Tiga Tiga	2	Ubah Hapus
14	manager56	Manager 55	2	Ubah Hapus
15	manager12	Manager 11	2	Ubah Hapus
16	manager11	Manager 11	2	Ubah Hapus

21. Laporkan hasil Praktikum-2.6 ini dan *commit* perubahan pada *git*.

Praktikum 2.7 – Relationships

One to One

Hubungan satu-ke-satu adalah tipe hubungan database yang sangat mendasar. Misalnya, suatu `Usermodel` mungkin dikaitkan dengan satu model `Levelmodel`. Untuk mendefinisikan hubungan ini, kita akan menempatkan `Levelmodel` metode pada model `Usermodel`. Metode tersebut `Levelmodel` harus memanggil `hasOne` metode tersebut dan mengembalikan hasilnya. Metode ini `hasOne` tersedia untuk model Anda melalui kelas dasar model `Illuminate\Database\Eloquent\Model`:



```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasOne;

You, 1 second ago | 1 author (You)
class UserModel extends Model
{
    public function level(): HasOne
    {
        return $this->hasOne(LevelModel::class);
    }
}
```

Mendefinisikan Kebalikan dari Hubungan *One-to-one*

Jadi, kita dapat mengakses model `Levelmodel` dari model `Usermodel` kita. Selanjutnya, mari kita tentukan hubungan pada model `Levelmodel` yang memungkinkan kita mengakses user. Kita dapat mendefinisikan kebalikan dari suatu `hasOne` hubungan menggunakan `belongsTo` metode:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class LevelModel extends Model
{
    public function user(): BelongsTo
    {
        return $this->belongsTo(UserModel::class);
    }
}
```

One to Many

Hubungan satu-ke-banyak digunakan untuk mendefinisikan hubungan di mana satu model adalah induk dari satu atau lebih model turunan. Misalnya, 1 kategori mungkin memiliki jumlah



barang yang tidak terbatas. Seperti semua hubungan Eloquent lainnya, hubungan satu-kebanyak ditentukan dengan mendefinisikan metode pada model Eloquent Anda:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class KategoriModel extends Model
{
    public function barang(): HasMany
    {
        return $this->hasMany(BarangModel::class, 'barang_id', 'barang_id');
    }
}
```

One to Many (Inverse) / Belongs To

Sekarang kita dapat mengakses semua barang, mari kita tentukan hubungan agar barang dapat mengakses kategori induknya. Untuk menentukan invers suatu `hasMany` hubungan, tentukan metode hubungan pada model anak yang memanggil `belongsTo` tersebut:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class BarangModel extends Model
{
    public function kategori(): BelongsTo
    {
        return $this->belongsTo(KategoriModel::class, 'kategori_id', 'kategori_id');
    }
}
```

1. Buka file model pada `UserModel.php` dan tambahkan scripnya menjadi seperti di bawah ini



```
class UserModel extends Model
{
    use HasFactory;

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
    /**
     * The attributes that are mass assignable.
     *
     * @var array
     */
    protected $fillable = ['level_id', 'username', 'nama', 'password'];

    public function level(): BelongsTo
    {
        return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
    }
}
```

Hasil :

```
app > Models > UserModel.php > PHP Intelephense > UserModel > level
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\Relations\BelongsTo;
8
9  6 references | 0 implementations
10 class UserModel extends Model
11 {
12     use HasFactory;
13
14     2 references
15     protected $table = 'm_user'; // Mendefinisikan nama tabel yang digunakan oleh model ini
16     2 references
17     protected $primaryKey = 'user_id'; // Mendefinisikan primary key dari tabel yang digunakan
18
19     4 references
20     protected $fillable = ['level_id', 'username', 'nama', 'password'];
21
22     0 references | 0 overrides
23     public function level(): BelongsTo
24     {
25         return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
26     }
27 }
```

- **LevelModel.php**

```
app > Models > LevelModel.php > PHP > LevelModel
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  1 reference | 0 implementations
9  class LevelModel extends Model
10 {
11     2 references
12     protected $table = "m_level"; // Mendefinisikan nama tabel yang digunakan oleh model ini
13     2 references
14     protected $primaryKey = "level_id"; // Mendefinisikan primary key
15 }
```




2. Buka file controller pada `UserController.php` dan ubah method *script* menjadi seperti di bawah ini

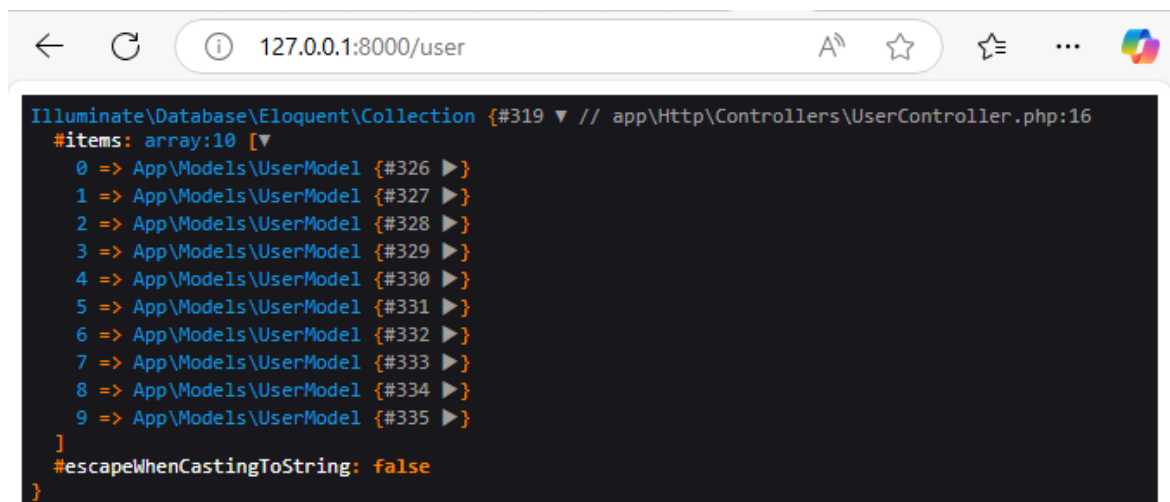
```
public function index()
{
    $user = UserModel::with('level')->get();
    dd($user);
}
```

Hasil :

```
//praktikum2.7 2
1 reference | 0 overrides
public function index(): never

    $user = UserModel::with('level')->get();
    dd(vars: $user);
```

3. Simpan kode program Langkah 2. Kemudian jalankan link pada *browser*, kemudian amati apa yang terjadi dan beri penjelasan dalam laporan



4. Buka file controller pada `UserController.php` dan ubah method *script* menjadi seperti di bawah ini

```
public function index()
{
    $user = UserModel::with('level')->get();
    return view('user', ['data' => $user]);
}
```

Hasil :



```
public function index(): Factory|View  
  
    $user = UserModel::with('level')->get();  
    return view('user', data: ['data' => $user]);
```

5. Buka file view pada user.blade.php dan ubah script menjadi seperti di bawah ini

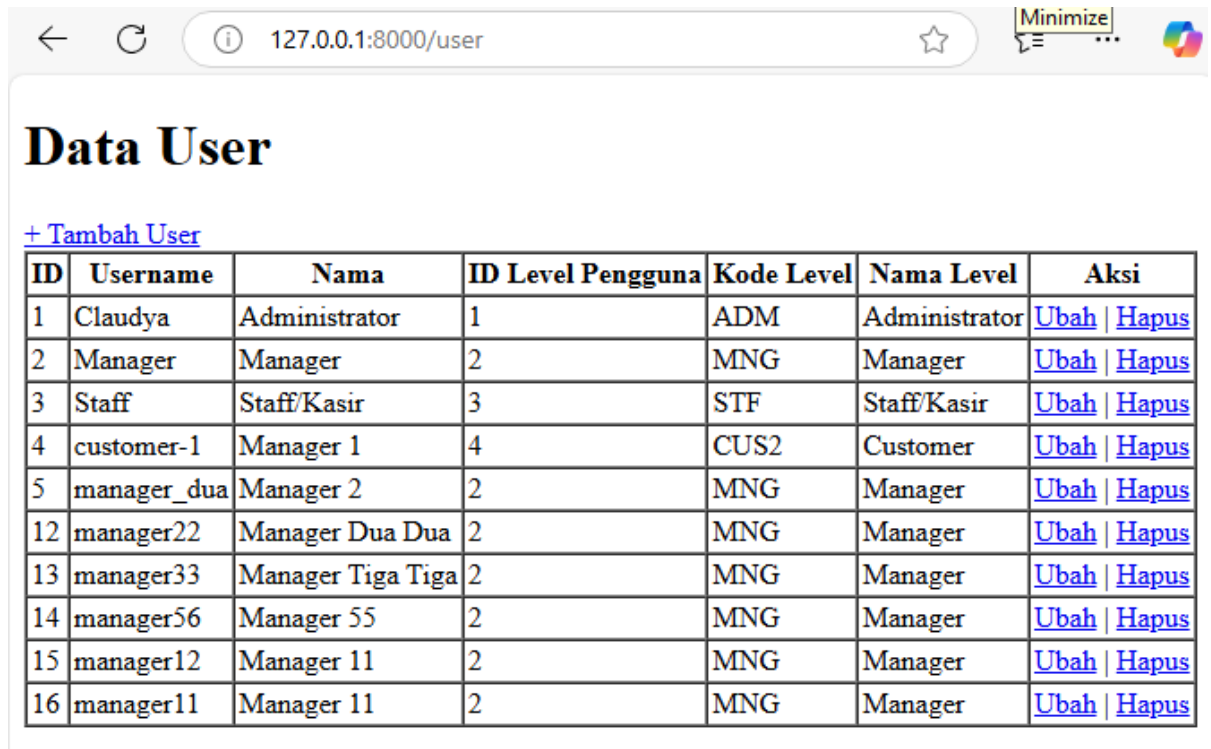
```
<body>  
<h1>Data User</h1>  
<a href="/user/tambah">+ Tambah User</a>  
<table border="1" cellpadding="2" cellspacing="0">  
    <tr>  
        <td>ID</td>  
        <td>Username</td>  
        <td>Nama</td>  
        <td>ID Level Pengguna</td>  
        <td>Kode Level</td>  
        <td>Nama Level</td>  
        <td>Aksi</td>  
    </tr>  
    @foreach ($data as $d)  
        <tr>  
            <td>{{ $d->user_id }}</td>  
            <td>{{ $d->username }}</td>  
            <td>{{ $d->nama }}</td>  
            <td>{{ $d->level_id }}</td>  
            <td>{{ $d->level->level_kode }}</td>  
            <td>{{ $d->level->level_nama }}</td>  
            <td><a href="/user/ubah/{{ $d->user_id }}">Ubah</a> | <a href="/user/hapus/{{ $d->user_id }}">Hapus</a></td>  
        </tr>  
    @endforeach  
</table>  
</body>
```

Hasil :

```
resources > views > user.blade.php > html > body > table > tr > td  
1 <!DOCTYPE html>  
2 <html>  
3 <head>  
4 <title>Data User</title>  
5 </head>  
6 <body>  
7 <h1>Data User</h1>  
8 <a href="/user/tambah">+ Tambah User</a>  
9 <table border="1" cellpadding="2" cellspacing="0">  
10 <tr>  
11 <th>ID</th>  
12 <th>Username</th>  
13 <th>Nama</th>  
14 <th>ID Level Pengguna</th>  
15 <th>Kode Level</th>  
16 <th>Nama Level</th>  
17 <th>Aksi</th>  
18 </tr>  
19 @foreach ($data as $d )  
20 <tr>  
21 <td>{{ $d->user_id }}</td>  
22 <td>{{ $d->username }}</td>  
23 <td>{{ $d->nama }}</td>  
24 <td>{{ $d->level_id }}</td>  
25 <td>{{ $d->level->level_kode }}</td>  
26 <td>{{ $d->level->level_nama }}</td>  
27 <td><a href="/user/ubah/{{ $d->user_id }}">Ubah</a> | <a href="/user/hapus/{{ $d->user_id }}">Hapus</a></td>  
28 </tr>  
29 @endforeach  
30 </table>  
31 </body>  
32 </html>
```



6. Simpan kode program Langkah 4 dan 5. Kemudian jalankan link pada *browser*, kemudian amati apa yang terjadi dan beri penjelasan dalam laporan



ID	Username	Nama	ID Level Pengguna	Kode Level	Nama Level	Aksi
1	Claudya	Administrator	1	ADM	Administrator	Ubah Hapus
2	Manager	Manager	2	MNG	Manager	Ubah Hapus
3	Staff	Staff/Kasir	3	STF	Staff/Kasir	Ubah Hapus
4	customer-1	Manager 1	4	CUS2	Customer	Ubah Hapus
5	manager_dua	Manager 2	2	MNG	Manager	Ubah Hapus
12	manager22	Manager Dua Dua	2	MNG	Manager	Ubah Hapus
13	manager33	Manager Tiga Tiga	2	MNG	Manager	Ubah Hapus
14	manager56	Manager 55	2	MNG	Manager	Ubah Hapus
15	manager12	Manager 11	2	MNG	Manager	Ubah Hapus
16	manager11	Manager 11	2	MNG	Manager	Ubah Hapus

7. Laporkan hasil Praktikum-2.7 ini dan *commit* perubahan pada *git*.

*** Sekian, dan selamat belajar ***